

Specyfikacja Wymagań Systemowych

FlatShare - Checkpoint 1

Grupa projektowa:

Jakub Rak, Bartosz Radomski, Sofia Kuzmenko, Piotr Wójcik

Spis treści

1	Wstęp	2
2	Historie Użytkownika (User Stories)	2
2.1	Aktor: Użytkownik (Wspólne)	2
2.2	Aktor: Lokator	2
2.3	Aktor: Właściciel Mieszkania	2
2.4	Aktor: Administrator	3
3	Diagramy Przypadków Użycia (Use Cases)	3
3.1	Moduł Ogłoszeń	3
3.2	Moduł Użytkownika i Komunikacji	4
3.3	Moduł Wynajmu i Płatności	5
3.4	Moduł moderacji	6
4	Szczegółowa specyfikacja (Deep Dive)	6
4.1	Przypadek: Wystawienie ogłoszenia	6
4.2	Przypadek: Rezerwacja pokoju (z płatnością)	6
4.3	Przypadek: Blokada użytkownika (Moderacja)	7
5	Diagramy Klas i Architektura (Class Diagrams)	9
5.1	Model domenowy (encje i powiązania)	9
5.2	Warstwa serwisów i repozytoriów (DIP/OCP)	9
5.3	Moduły i komunikacja	9
5.4	Kluczowe klasy i ich odpowiedzialność	10
5.5	Wymiana informacji między modułami (Inter-module communication)	10

1 Wstęp

Dokument zawiera specyfikację wymagań funkcjonalnych oraz projekt architektury systemu. Celem systemu jest połączenie właścicieli mieszkań z potencjalnymi lokatorami poprzez mechanizmy dopasowania preferencji oraz bezpośrednią komunikację. System jest podzielony na niezależne moduły, które komunikują się poprzez warstwę serwisową.

2 Historie Użytkownika (User Stories)

Poniżej przedstawiono wymagania biznesowe zagregowane według aktorów systemu.

2.1 Aktor: Użytkownik (Wspólne)

- **US-Sys1:** Jako Użytkownik, chcę bezpiecznie logować się do systemu, aby uzyskać dostęp do swojego konta.
- **US-Sys2:** Jako Użytkownik, chcę mieć możliwość resetu hasła, aby odzyskać dostęp do konta w przypadku jego zapomnienia.

2.2 Aktor: Lokator

- **US-L1:** Jako Lokator, chcę przeglądać i filtrować ogłoszenia (wg ceny, lokalizacji), aby szybko znaleźć pokój spełniający moje kryteria.
- **US-L2:** Jako Lokator, chcę zdefiniować swoje preferencje (np. zwierzęta, palenie), aby system mógł podpowiadać mi najlepiej dopasowane oferty.
- **US-L3:** Jako Lokator, chcę wysłać wiadomość do właściciela, aby dopytać o szczegóły wynajmu.
- **US-L4:** Jako Lokator, chcę wysłać prośbę o rezerwację pokoju, aby sfinalizować proces wynajmu.
- **US-L5:** Jako Lokator, chcę opłacić rezerwację przez zewnętrzny system płatności, aby potwierdzić wynajem.

2.3 Aktor: Właściciel Mieszkania

- **US-W1:** Jako Właściciel, chcę wystawić ogłoszenie ze zdjęciami i opisem, aby dotrzeć do potencjalnych najemców.
- **US-W2:** Jako Właściciel, chcę określić preferowany profil lokatora (np. student, osoba pracująca), aby uniknąć nieporozumień.
- **US-W3:** Jako Właściciel, chcę zarządzać dostępnością ogłoszenia (publikować/ukrywać/archiwizować), gdy mieszkanie zostanie wynajęte.
- **US-W4:** Jako Właściciel, chcę zaakceptować lub odrzucić rezerwację, aby kontrolować kto wynajmuje pokój.
- **US-W5:** Jako Właściciel, chcę włączyć opcję *Instant Book*, aby automatyzować rezerwacje bez ręcznej akceptacji.

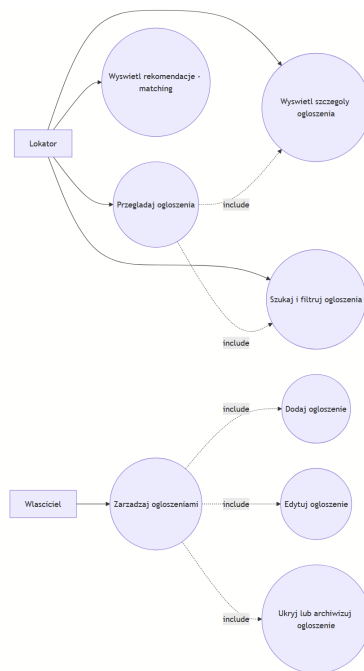
2.4 Aktor: Administrator

- **US-A1:** Jako Admin, chcę mieć możliwość blokowania użytkowników łamiących regulamin, aby utrzymać porządek w serwisie.
- **US-A2:** Jako Admin, chcę przeglądać zgłoszone naruszenia w panelu administracyjnym, aby szybko reagować na nadużycia.

3 Diagramy Przypadków Użycia (Use Cases)

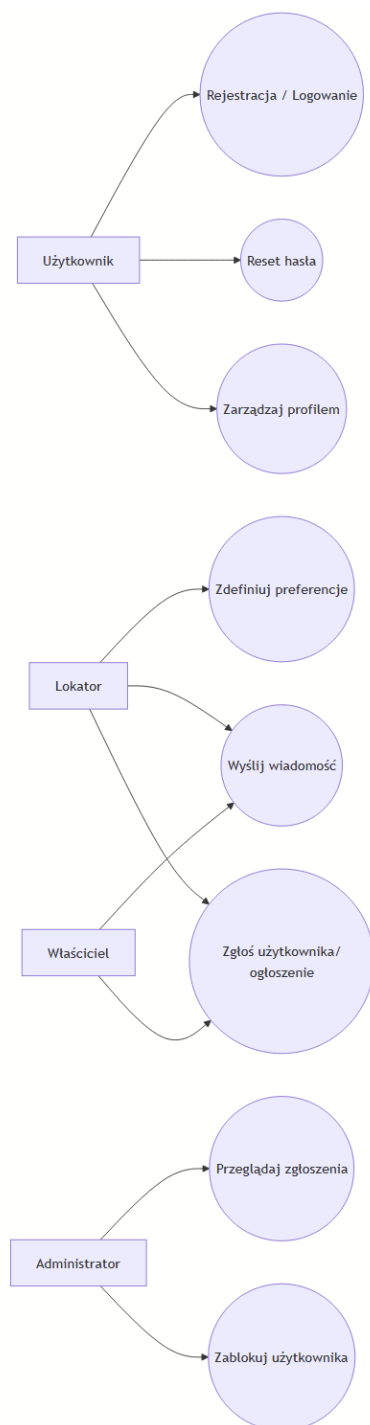
Funkcjonalności zostały podzielone na logiczne moduły, co ułatwia analizę systemu. Diagramy przypadków użycia stanowią ilustrację do wstępów poszczególnych grup funkcjonalności.

3.1 Moduł Ogłoszeń



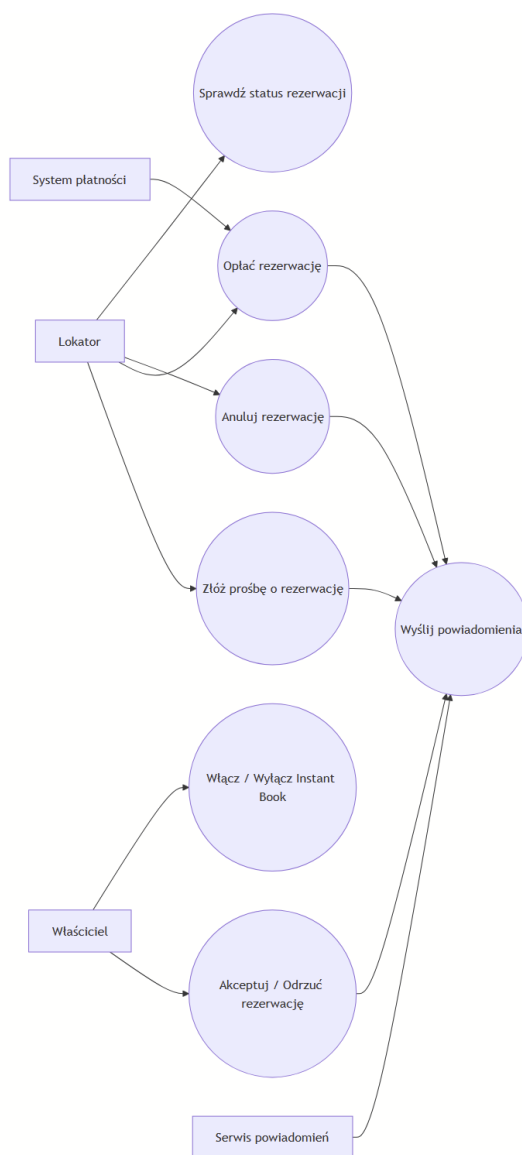
Rysunek 1: Diagram przypadków użycia - Moduł Ogłoszeń

3.2 Moduł Użytkownika i Komunikacji



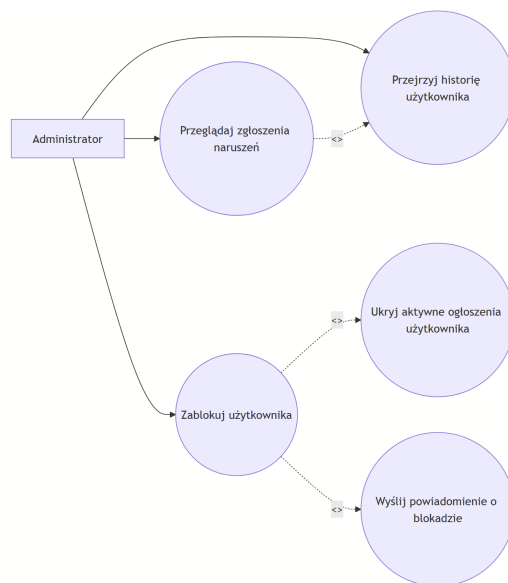
Rysunek 2: Diagram przypadków użycia - Moduł Użytkownika i Komunikacji

3.3 Moduł Wynajmu i Płatności



Rysunek 3: Diagram przypadków użycia - Moduł Wynajmu/Rezerwacji i Płatności (realizacja US-L4, US-L5, US-W4, US-W5)

3.4 Moduł moderacji



Rysunek 4: Moderacja — przypadki użycia

4 Szczegółowa specyfikacja (Deep Dive)

Szczegółowy opis kluczowych przypadków użycia wg szablonu Cockburna (główne scenariusze oraz kluczowe alternatywy).

4.1 Przypadek: Wystawienie ogłoszenia

Nazwa przypadku	Wystawienie ogłoszenia (Create Listing)
Aktor główny	Właściciel Mieszkania
Aktorzy pomocniczy	System Przechowywania Plików (File Storage), (opcjonalnie) System Płatności
Warunki wstępne	Użytkownik zalogowany, posiada rolę Właściciela.
Główny scenariusz	1. Właściciel wybiera opcję “Dodaj ogłoszenie”. 2. System prosi o dane (tytuł, opis, cena, lokalizacja) oraz zdjęcia. 3. Właściciel wprowadza dane i dodaje zdjęcia. 4. System waliduje dane (np. $cena > 0$, wymagane pola). 5. System zapisuje ogłoszenie (status: <i>DRAFT</i> lub <i>UNDER_REVIEW</i> zgodnie z polityką). 6. (Jeśli dotyczy) System publikuje ogłoszenie (status: <i>ACTIVE</i>).
Scenariusze alternatywne	4a. Błąd walidacji: System wyświetla błędy i prosi o poprawę. 5a. Limit darmowych publikacji: Jeśli użytkownik przekroczył limit darmowych ogłoszeń, system przechodzi do płatności i dopiero po sukcesie umożliwia publikację.

4.2 Przypadek: Rezerwacja pokoju (z płatnością)

Nazwa przypadku	Rezerwacja pokoju (Book a Room)
Aktor główny	Lokator
Aktorzy pomocniczy	Właściciel, System Płatności (zewnętrzny), Serwis Powiadomień
Warunki wstępne	Lokator zalogowany, ogłoszenie ma status <i>ACTIVE</i> , pokój dostępny dla wybranego okresu.
Główny scenariusz	1. Lokator na stronie ogłoszenia klika "Rezerwuj". 2. System pokazuje podsumowanie kosztów, regulamin i prosi o potwierdzenie. 3. Lokator potwierdza chęć rezerwacji. 4. System tworzy rezerwację ze statusem <i>PENDING_APPROVAL</i> i wysyła powiadomienie do Właściciela. 5. Właściciel akceptuje rezerwację. System zmienia status rezerwacji na <i>PENDING_PAYMENT</i>. 6. System przekierowuje Lokatora do Systemu Płatności. 7. System Płatności potwierdza płatność. System zmienia status rezerwacji na <i>CONFIRMED</i>. 8. System aktualizuje dostępność (np. blokuje termin / ustawia ogłoszenie jako niedostępne dla konfliktowych rezerwacji). 9. Serwis Powiadomień wysyła potwierdzenia do Lokatora i Właściciela.
Scenariusze alternatywne	4a. Instant Book: Jeśli Właściciel włączył <i>Instant Book</i>, system pomija akceptację: rezerwacja powstaje od razu jako <i>PENDING_PAYMENT</i> i przechodzi do kroku 6. 5a. Odrzucenie: Właściciel odrzuca rezerwację. System ustawia status <i>REJECTED</i> i powiadamia Lokatora. 5b. Brak reakcji: Właściciel nie odpowiada w ustalonym czasie. System ustawia status <i>EXPIRED</i> i powiadamia Lokatora. 7a. Błąd płatności: Płatność nieudana. System ustawia status <i>PAYMENT_FAILED</i> i umożliwia ponowienie lub anulowanie. Ogłoszenie zajęte: W międzyczasie termin stał się niedostępny (kolizja). System anuluje proces i wyświetla komunikat.

4.3 Przypadek: Blokada użytkownika (Moderacja)

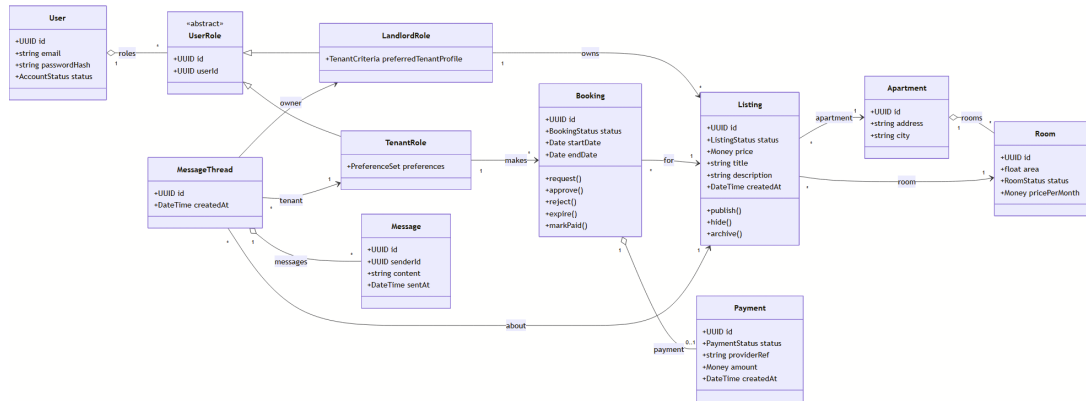
Nazwa przypadku	Blokada użytkownika (Ban User)
Aktor główny	Administrator
Warunki wstępne	Admin zalogowany do panelu administracyjnego.

Główny scenariusz	1. Admin przegląda listę zgłoszeń naruszeń. 2. Admin analizuje historię zgłoszonego użytkownika. 3. Admin wybiera opcję “Zablokuj konto” i podaje powód. 4. System zmienia status użytkownika na <i>BLOCKED</i>. 5. Serwis Powiadomień wysyła wiadomość email z informacją o blokadzie. 6. Wszystkie aktywne ogłoszenia użytkownika są ukrywane.
Scenariusze alternatywne	3a. Odrzucenie zgłoszenia: Admin uznaje zgłoszenie za bezzasadne i zamyka sprawę bez blokady.

5 Diagramy Klas i Architektura (Class Diagrams)

Poniższe diagramy prezentują strukturę klas zaprojektowaną w celu realizacji funkcjonalności zdefiniowanych w diagramach przypadków użycia. Projekt rozdziela modele danych (encje domenowe) od logiki (serwisy), zgodnie z zasadami SOLID.

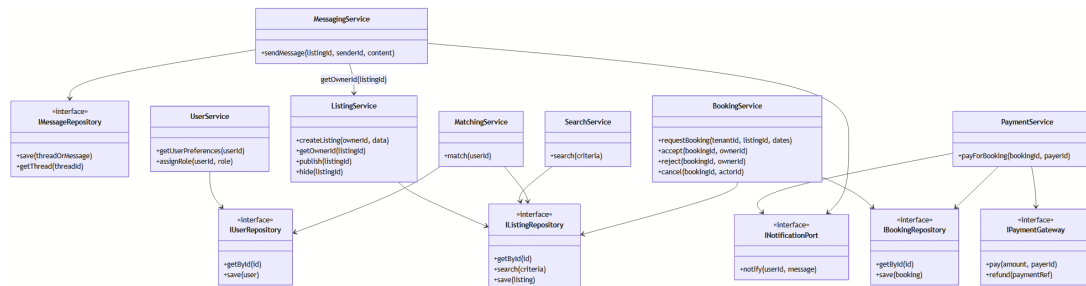
5.1 Model domenowy (encje i powiązania)



Rysunek 5: Diagram klas - model domenowy (User + Role, Listing, Apartment/Room, Booking, Payment, Messaging)

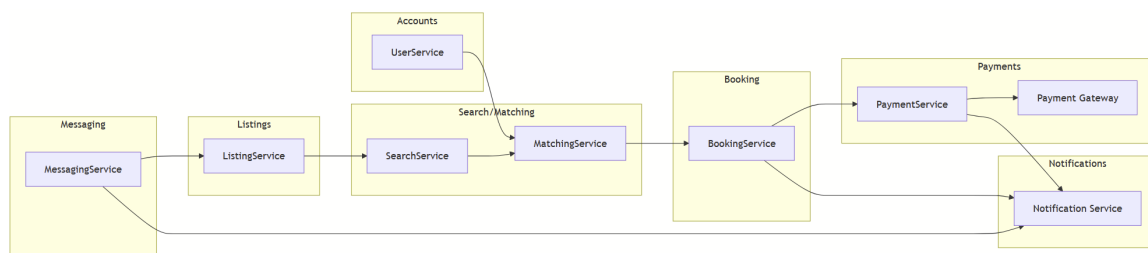
Uwaga projektowa (spójność domeny). Rezerwacja (Booking) jest zawsze tworzona przez Lokatora (rola TenantRole) i dotyczy konkretnego ogłoszenia (Listing). Z tego powodu w modelu domenowym Booking posiada powiązania do TenantRole oraz Listing.

5.2 Warstwa serwisów i repozytoriów (DIP/OCP)



Rysunek 6: Diagram klas - serwisy, repozytoria i porty integracyjne (płatności, powiadomienia)

5.3 Moduły i komunikacja



Rysunek 7: Podział na moduły i kanały komunikacji (inter-module communication)

5.4 Kluczowe klasy i ich odpowiedzialność

Zgodnie z wymaganiami projektowymi, system opiera się na obiektach domenowych oraz serwisach zarządzających logiką.

1. Użytkownicy i Role (Kompozycja):

- Zamiast dziedziczenia ról od `User`, zastosowano kompozycję: `User` posiada przypisane obiekty ról (np. `TenantRole`, `LandlordRole`).
- `User` przechowuje dane uwierzytelniające oraz status konta.
- Role definiują uprawnienia i kontekst danych: preferencje należą do roli Lokatora, a zarządzanie mieszkaniem/ogłoszeniami do roli Właściciela.

2. Ogłoszenia (Listing) oraz struktura lokalu:

- `Listing` reprezentuje ogłoszenie powiązane z `Apartment` i `Room`.
- **Status ogłoszenia:** `Listing.status` (np. `ACTIVE`, `HIDDEN`, `ARCHIVED`) pozwala zarządzać dostępnością i publikacją.

3. Wynajem/Rezerwacja i Płatność:

- `Booking` reprezentuje proces rezerwacji i przechowuje status (np. `PENDING_APPROVAL`, `PENDING_PAYMENT`, `CONFIRMED`).
- `Payment` przechowuje wynik integracji z bramką płatności (referencja dostawcy, status).

4. Komunikacja:

- Zamiast pojedynczych wiadomości bez kontekstu, zastosowano `MessageThread` (wątek) oraz `Message` (wiadomość), co upraszcza utrzymanie historii rozmów i powiązanie z ogłoszeniem.

5.5 Wymiana informacji między modułami (Inter-module communication)

System realizuje komunikację między niezależnymi modułami poprzez warstwę serwisową i porty integracyjne (interfejsy), aby moduły nie były bezpośrednio ze sobą sprzężone.

• Moduł Komunikacji ← Moduł Ogłoszeń

Aby zrealizować przypadek użycia „Wysyłanie wiadomości” w kontekście konkretnego ogłoszenia, moduł komunikacji musi ustalić adresata.

Wymagana interakcja: `MessagingService` wywołuje `ListingService.getOwnerId(listingId)` aby pobrać identyfikator właściciela.

• Moduł Dopasowań ← Moduł Użytkownika (Preferencje)

Aby dopasować ogłoszenia do Lokatora, `MatchingService` potrzebuje preferencji Lokatora.

Wymagana interakcja: `MatchingService` pobiera dane poprzez `UserService.getUserPreferences(userId)`.

• Moduł Wynajmu → Moduł Płatności (integracja zewnętrzna)

Aby potwierdzić rezerwację, moduł wynajmu inicjuje płatność w zewnętrznej bramce.

Wymagana interakcja: `PaymentService.payForBooking(bookingId, payerId)` korzysta z portu `IPaymentGateway`.

• Moduł Wynajmu/Płatności → Moduł Powiadomień

Po kluczowych zdarzeniach (złożenie rezerwacji, akceptacja/odrzućenie, płatność) system powiadamia użytkowników.

Wymagana interakcja: `BookingService` i `PaymentService` korzystają z portu `INotificationPort`.